Project Design Phase-II
Technology Stack (Architecture & Stack)

Field	Details
Project Design Phase	**Phase-II**
Project Name	**Serene-Study: AI-Based Exam Anxiety Detector**
Date	**March 7, 2026**
Team ID	**GACSE-109**
Maximum Marks	**4 Marks**

1. Technical Architecture

The **Serene-Study** ecosystem is built on a modern, decoupled **Client-Server architecture**. The system is designed to handle high-concurrency student reflections by offloading heavy computation (BERT inference and Gemini generation) to a specialized FastAPI backend.

Data flows from the **Streamlit Frontend** through a secure REST API to the **FastAPI Engine**, which orchestrates state management between the local **SQLite** cache and **Firebase Firestore** cloud storage.

Table 1: Detailed Component Stack

S.No	Component	Description	Technology
1.	**User Interface**	Responsive web dashboard with dual-mode (Student/Admin) access.	Streamlit, Python, Custom CSS
2.	**Application Logic-1**	Natural Language Processing (NLP) Pipeline: Cleaning & Tokenization.	NLTK, Transformers (Hugging Face)
3.	**Application Logic-2**	Predictive Engine: Categorizing text into 3 anxiety levels.	PyTorch, BERT-base-uncased
4.	**Application Logic-3**	Generative Engine: Creating personality-driven coaching reports.	Google Gemini 1.5 Pro
5.	**Database**	Lightweight local storage for session logs and error tracking.	SQLite (anxiety_logs.db)

6.	**Cloud Database**	Real-time NoSQL storage for campus-wide trend analysis.	Firebase Firestore
7.	**File Storage**	Optimized storage for pre-trained model weights (.pt files).	Hugging Face Spaces / Docker Volumes
8.	**External API-1**	Generative AI integration for dynamic coping strategy generation.	Google Gemini API
9.	**External API-2**	Identity and Access Management for secure data persistence.	Firebase Admin SDK
10.	**ML Model**	Transformer-based model fine-tuned on student anxiety datasets.	BERT (Bidirectional Encoder Representations)
11.	**Infrastructure**	Containerized deployment environment for cloud scaling.	Docker, Render, Streamlit Cloud

2. Application Characteristics

Table 2: Architectural Justification

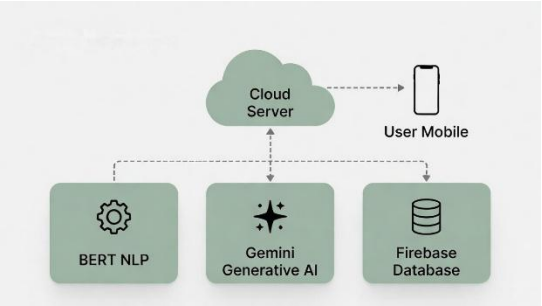
S.No	Characteristics	Description	Technology
1.	**Open-Source**	Utilizing industry-standard frameworks for rapid, cost-effective development.	PyTorch, FastAPI, Streamlit
2.	**Security**	Multi-layer protection including API Key masking and Firebase IAM.	SHA-256 Encryption, IAM Controls
3.	**Scalability**	Micro-service style separation of Frontend and Backend for independent scaling.	Docker, 3-Tier Architecture
4.	**Availability**	High uptime ensured through cloud-native auto-scaling and health probes.	Render Health Checks, Load Balancers
5.	**Performance**	Asynchronous request handling to maintain <1.2s response times.	Python Asyncio, CDN (Cloudflare)

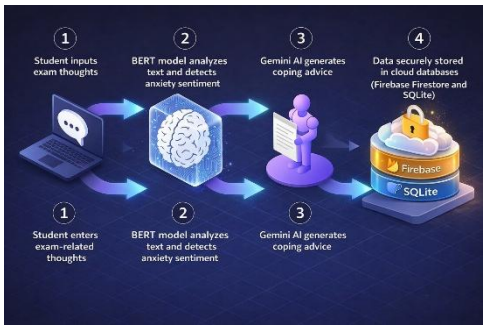
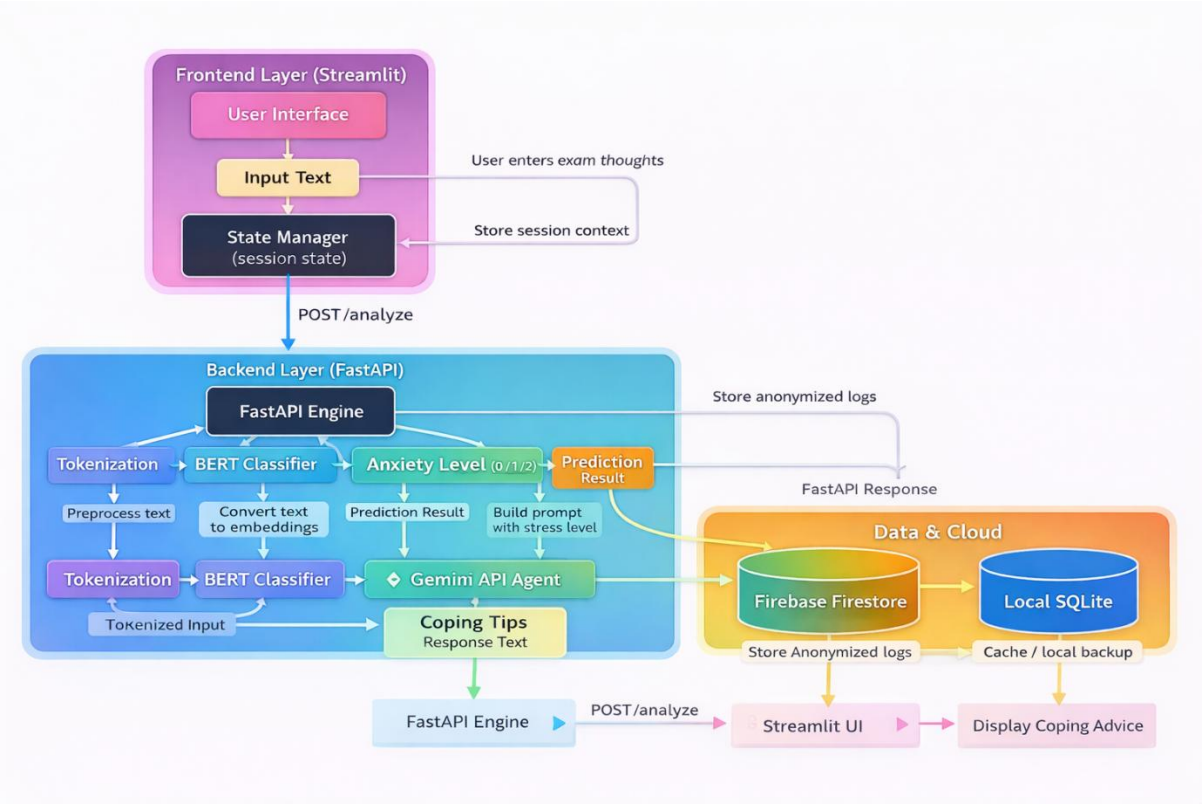
3. Infrastructural Demarcation

- **Local Environment:** VS Code with Python 3.10 virtual environment for development and testing.
- **Cloud Training Layer:** Google Colab utilizing NVIDIA T4 GPUs for BERT fine-tuning.
- **Production Deployment:** * **Backend:** FastAPI containerized via Docker on Render.

- **Frontend:** Streamlit application hosted on Streamlit Community Cloud.
- **Database:** Firebase Firestore (Global Instance).

4. Architectural Design Prompts (For Image-Gen AI)





5. Technical DFD/Architecture